

POLITECNICO
MILANO 1863

Scuola di Ingegneria Industriale e dell'Informazione

Corso di Ingegneria del Software — Prova Finale

Mesos

Specifica del Protocollo di Rete

Architettura Client-Server — Socket TCP / Java RMI

Docente: Alessandro Margara

Studenti: Andrea Cacioppo
Andrea Carbone
Gabriele Buca
Francesco Botton

Anno Accademico 2025/2026

Indice

1 Panoramica del Protocollo **3**
1.1 Decisioni Progettuali . . . 3
1.2 Confronto dei Trasporti . . . 3

2 Architettura **3**
2.1 Struttura dei Package . . . 3
2.2 Architettura di Sistema . . . 5

3 Protocollo Messaggi **6**
3.1 Gerarchia dei Messaggi . . . 6
3.2 Request: Client → Server . . . 6
3.3 Update: Server → Client . . . 7

4 Ciclo di Vita della Connessione **8**
4.1 Sequenza di Avvio . . . 8
4.2 Ciclo di Vita di un'Azione . . . 8

5 Heartbeat e Disconnessione **10**
5.1 Meccanismo Ping-Pong . . . 10
5.2 Flusso di Disconnessione . . . 11
5.3 Sospensione e Vittoria per Abbandono . . . 11

6 Protocollo di Riconnessione **13**
6.1 Flusso di Riconnessione Server-Side . . . 13
6.2 Auto-Riconnessione Lato Client . . . 14

7 Recupero da Crash del Server **15**
7.1 Backup JSON e Ripristino . . . 15

8 Interfacce di Connessione Lato Server **16**
8.1 Gerarchia ClientConnection . . . 16
8.2 API ClientConnection . . . 16

9 Interfacce di Connessione Lato Client **17**
9.1 ServerConnection . . . 17
9.2 Implementazioni . . . 17
9.3 VirtualServer — Facciata per la View . . . 17

10 Protocollo TCP Socket **19**
10.1 Layout degli Stream . . . 19
10.2 Dispatch Lato Server e Lato Client . . . 19

11 Interfacce Remote RMI **20**

11.1 Relazioni tra le Interfacce 20
11.2 RMIGameServer — Punto di Ingresso 20
11.3 RMIClientHandlerInterface — Azioni di Gioco 20
11.4 RMIClientInterface — Callback Client 21

12 Thread Safety **22**

13 Riferimenti ai File Sorgente **22**

1 Panoramica del Protocollo

Il protocollo di rete AM54 definisce le regole di comunicazione tra uno o più client di gioco e un server centrale. Il sistema implementa un **doppio strato di trasporto**: sia Socket TCP (object streams) sia Java RMI sono supportati simultaneamente, condividendo la stessa gerarchia di messaggi e lo stesso pattern Observer.

1.1 Decisioni Progettuali

Dual Transport Lo stesso protocollo logico è disponibile su TCP (porta 12345) e RMI (porta 1099). La scelta è trasparente alla logica di gioco.

Dispatch polimorfico I messaggi portano la propria logica di esecuzione: le *Request* implementano `execute(Controller, String, ClientConnection)`; gli *Update* implementano `applyTo(ClientModel)`. Non sono necessari costrutti `switch`/`instanceof`.

Observer Pattern La `VirtualView` lato server osserva il modello `Game` e traduce ogni evento in un `Update` trasmesso a tutti i client.

Aggiornamenti differenziali La maggior parte degli update trasporta solo i dati cambiati. Lo `StateChangedUpdate` contiene lo stato completo ed è inviato ad ogni transizione di fase e alla riconnessione.

1.2 Confronto dei Trasporti

Aspetto	Socket TCP	RMI
Porta	12345	1099
Serializzazione	Java Object Streams	Protocollo RMI
Entry point	`new Socket(host, 12345)`	`getRegistry(host, 1099)`
Consegna update	Thread background legge ObjectInputStream	Server chiama stub del client
Heartbeat	PingRequest / PongResponse	`ping()` / `pong()` remoti
Disconnessione	`IOException` su socket	`RemoteException`

2 Architettura

2.1 Struttura dei Package

`network/client/`

- Client.java — Singleton hub del client
- ServerConnection.java — Astrazione trasporto (interfaccia)
- SocketServerConnection.java — Implementazione TCP
- RMIServerConnection.java — Implementazione RMI + target callback

- ServerHandler.java — Thread lettura background TCP
- ClientModel.java — Stato locale lato client
- VirtualServer.java — Facciata azioni per la View

`network/server/` - Server.java — Entry point (ServerSocket TCP + registry RMI)
- ClientConnection.java — Astrazione trasporto (interfaccia)
- SocketClientHandler.java — Thread TCP per singolo client
- RMIClientHandler.java — Handler RMI per singolo client
- VirtualView.java — Osservatore Game → broadcast
- PersistenceManager.java — Backup JSON per crash recovery

`network/rmi/` - RMIGameServer.java — Interfaccia remota d'ingresso
- RMIClientInterface.java — Interfaccia callback client
- RMIClientHandlerInterface.java — Interfaccia handler server

`network/messages/` - ClientMessage.java — Interfaccia marcatore radice (Serializable)
- Request.java — Client → Server (*sealed*, 9 sottotipi)
- Update.java — Server → Client (*sealed*, 16 sottotipi)

2.2 Architettura di Sistema

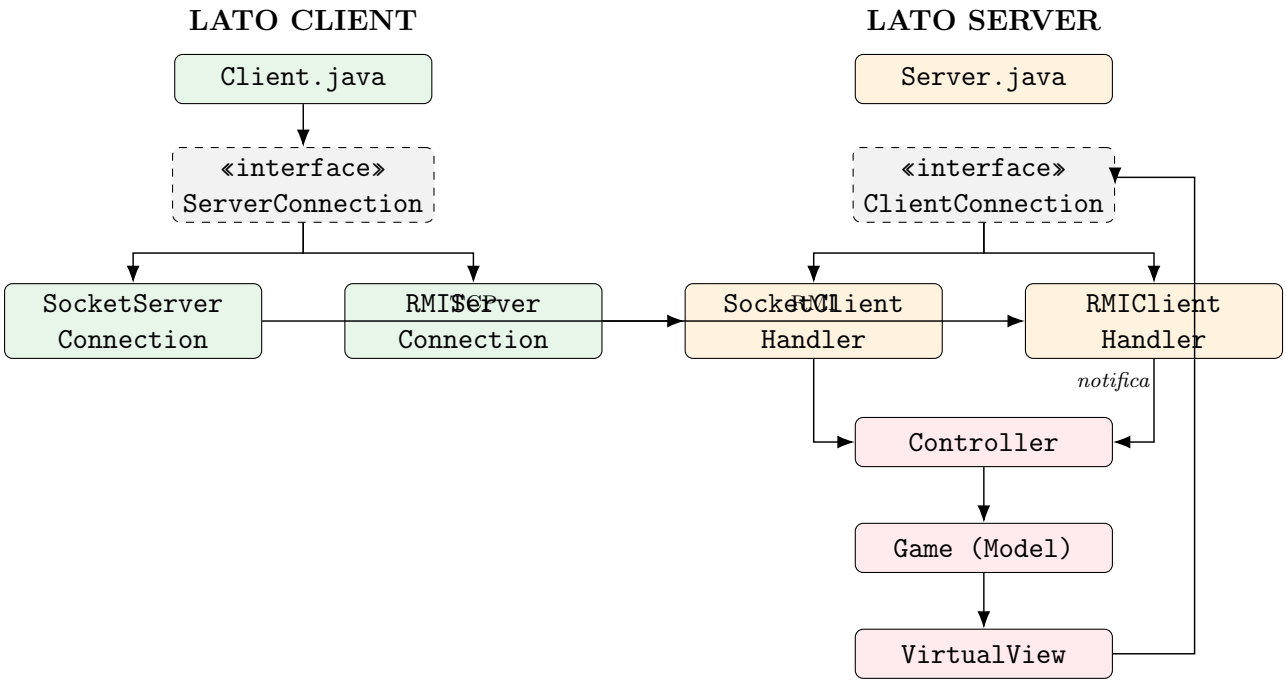


Figura 1: Architettura generale del sistema

3 Protocollo Messaggi

3.1 Gerarchia dei Messaggi

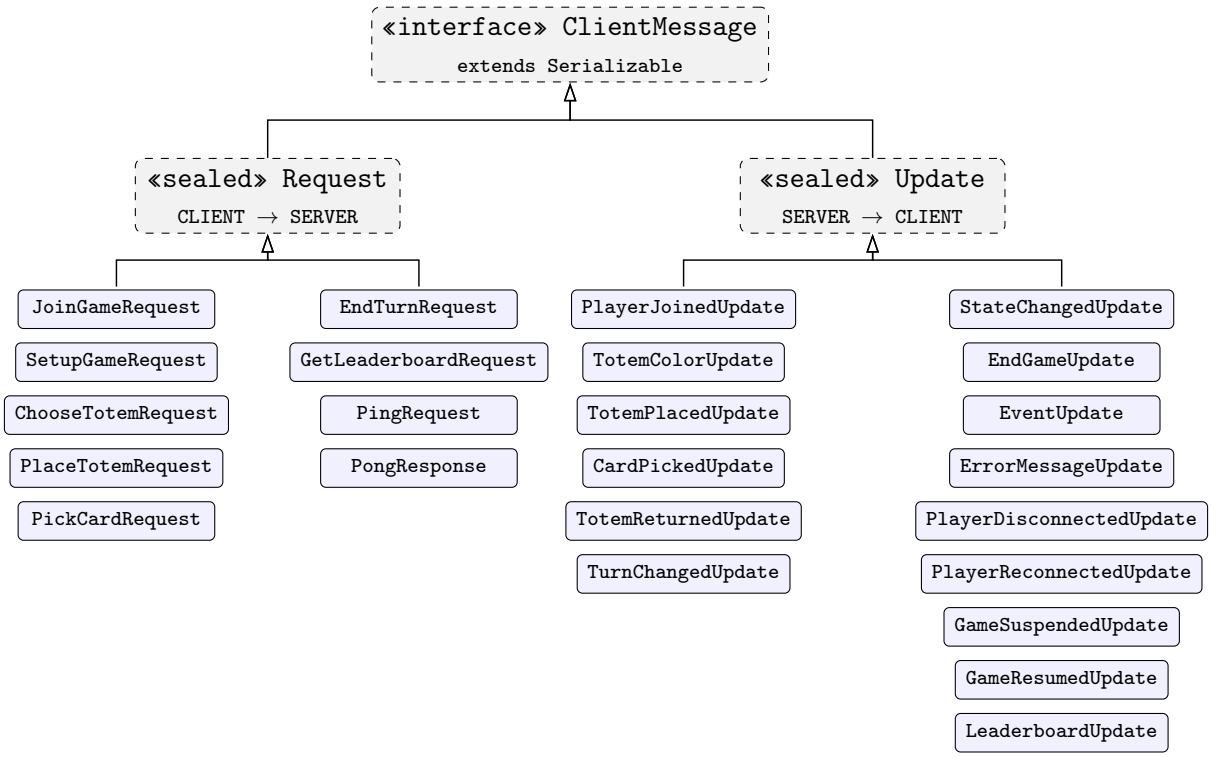


Figura 2: Gerarchia dei messaggi di rete

3.2 Request: Client → Server

Messaggio	Campi	Scopo
`JoinGameRequest`	`String nickname`	Connessione iniziale / rientro
`SetupGameRequest`	—	Segnala prontezza ad iniziare
`ChooseTotemRequest`	`TotemColors color`	Scelta colore totem
`PlaceTotemRequest`	`int offerPosition`	Posiziona totem su offering tile
`PickCardRequest`	`String row, int index`	Preleva carta da una fila
`EndTurnRequest`	—	Termina il turno corrente
`GetLeaderboardRequest`	`int numPlayers`	Richiede la classifica
`PingRequest`	`long timestamp`	Heartbeat (è anche Update)
`PongResponse`	`long timestamp`	Risposta all'heartbeat

3.3 Update: Server → Client

Messaggio	Campi principali	Scopo
`PlayerJoinedUpdate`	`String nickname`	Nuovo giocatore connesso
`TotemColorUpdate`	`String nick, TotemColors c`	Colore totem scelto
`TotemPlacedUpdate`	`String nick, int pos`	Totem posizionato
`CardPickedUpdate`	nick, row, idx, food, magic, offering	Carta prelevata
`TotemReturnedUpdate`	`String nick, int pos, food, aura`	Totem restituito
`TurnChangedUpdate`	`String nickname`	Cambio turno
`StateChangedUpdate`	state, era, round, players, turnOrder, offering, 6 liste carte	Snapshot completo
`EndGameUpdate`	`List winners, Map scores, int num`	Partita terminata
`EventUpdate`	name, players, 6 liste	Risoluzione evento
`ErrorMessageUpdate`	`String message`	Errore / mossa invalida
`PlayerDisconnectedUpdate`	`String nickname`	Giocatore disconnesso
`PlayerReconnectedUpdate`	`String nickname`	Giocatore riconnesso
`GameSuspendedUpdate`	—	Partita sospesa (1 attivo)
`GameResumedUpdate`	—	Partita ripresa ($\geq$2)
`LeaderboardUpdate`	`int pos, List entries, int num`	Risultato classifica
`PingRequest`	`long timestamp`	Heartbeat server → client

4 Ciclo di Vita della Connessione

4.1 Sequenza di Avvio

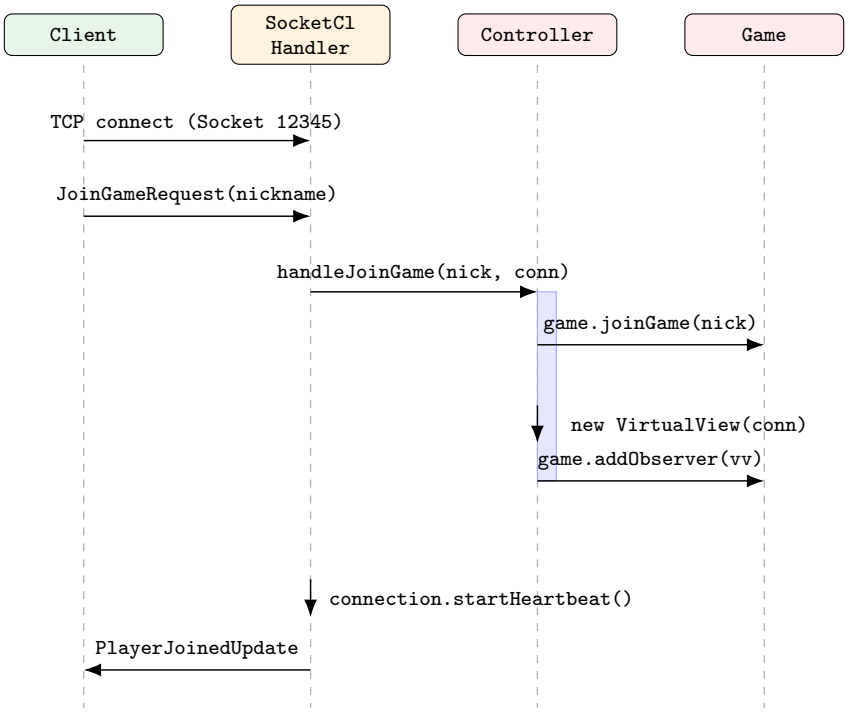


Figura 3: Sequenza di avvio: dal TCP connect alla notifica in lobby

4.2 Ciclo di Vita di un'Azione

Il diagramma traccia l'azione *PlaceTotem* dal click sulla UI all'aggiornamento di tutti i client connessi.

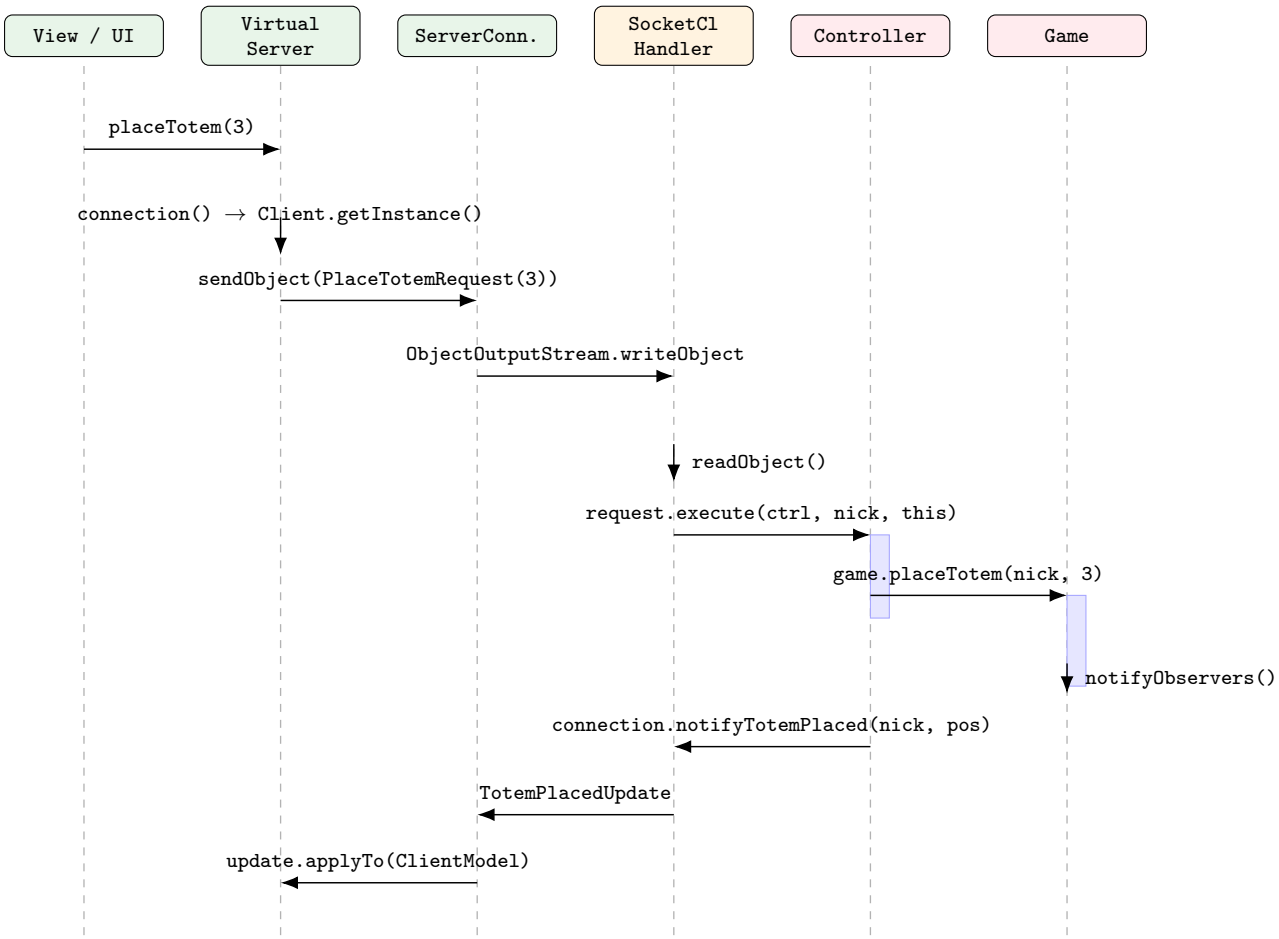


Figura 4: Ciclo di vita completo di un'azione (PlaceTotemRequest)

5 Heartbeat e Disconnessione

5.1 Meccanismo Ping-Pong

Il server usa uno `ScheduledExecutorService` per inviare un ping ogni **5 000 ms**. Se entro **15 000 ms** non arriva risposta, la connessione è dichiarata persa.

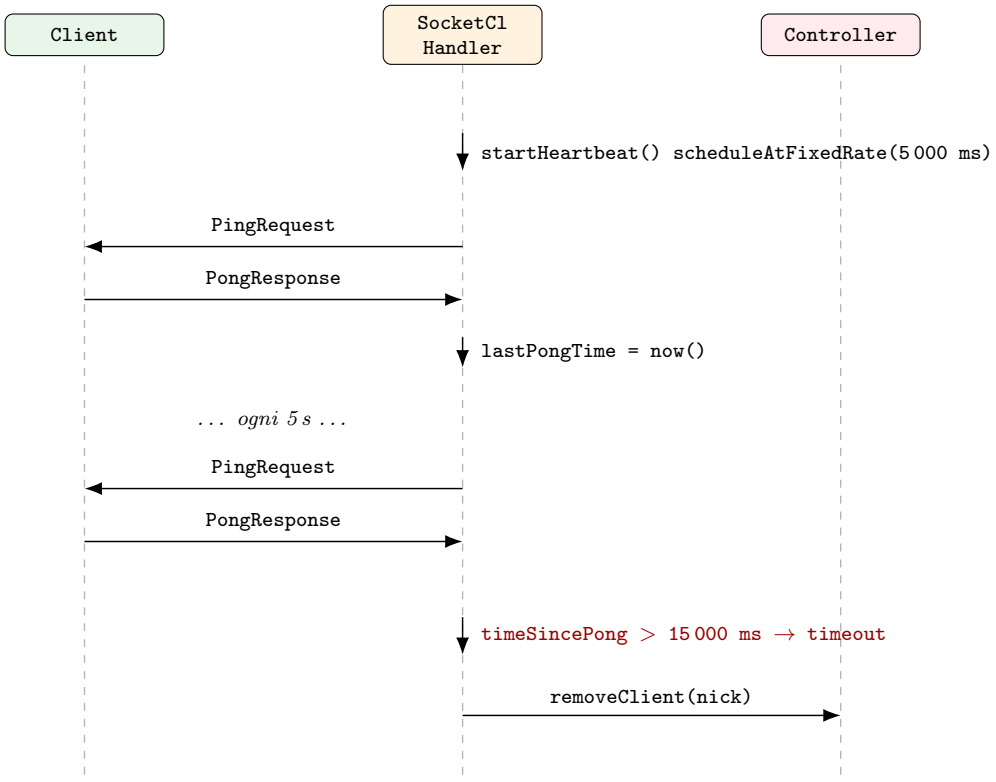


Figura 5: Heartbeat ping-pong e rilevamento timeout

5.2 Flusso di Disconnessione

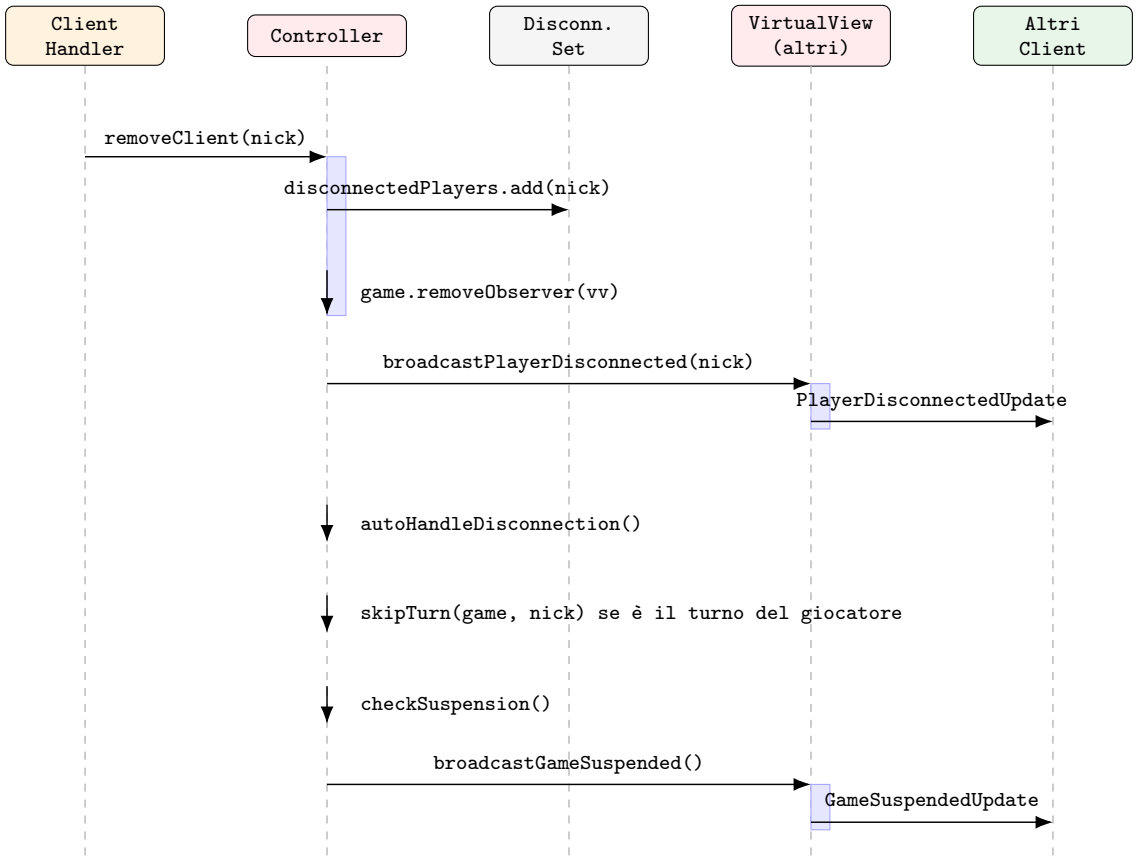


Figura 6: Cascata di disconnessione durante una partita attiva

5.3 Sospensione e Vittoria per Abbandono

Con un solo giocatore attivo il server avvia un countdown di 60 s. Allo scadere il giocatore rimasto vince per abbandono.

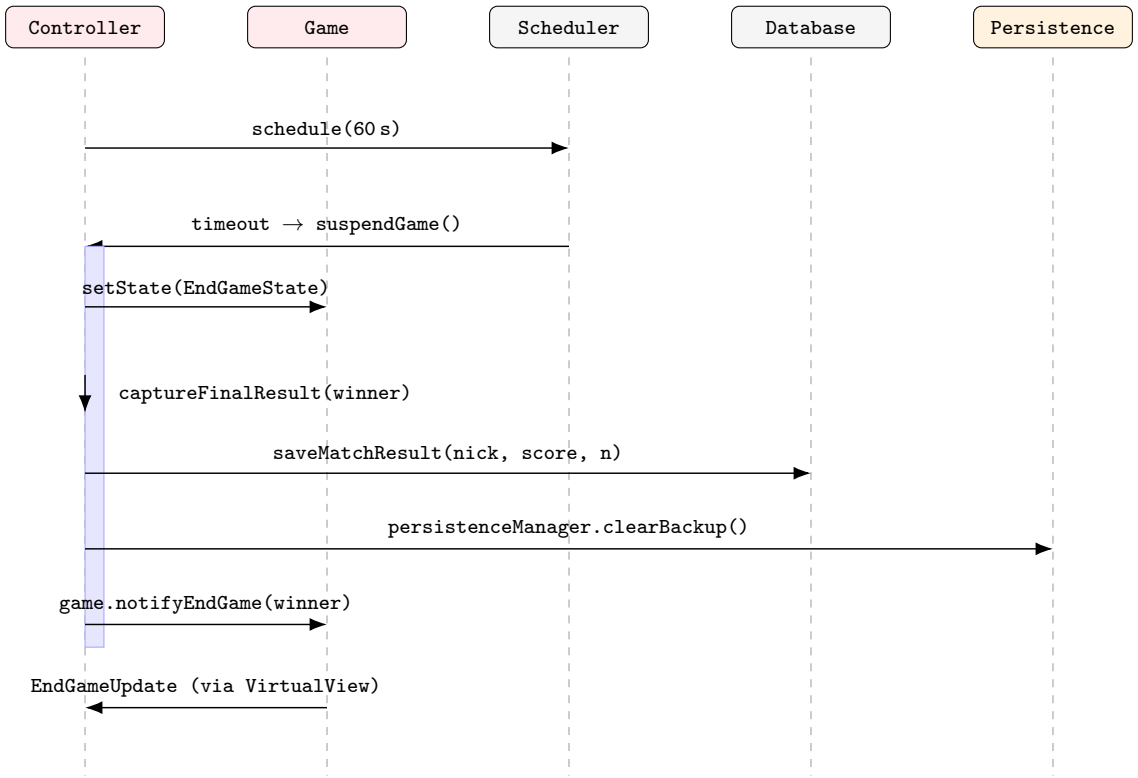


Figura 7: Timeout da sospensione e vittoria per abbandono

6 Protocollo di Riconnessione

6.1 Flusso di Riconnessione Server-Side

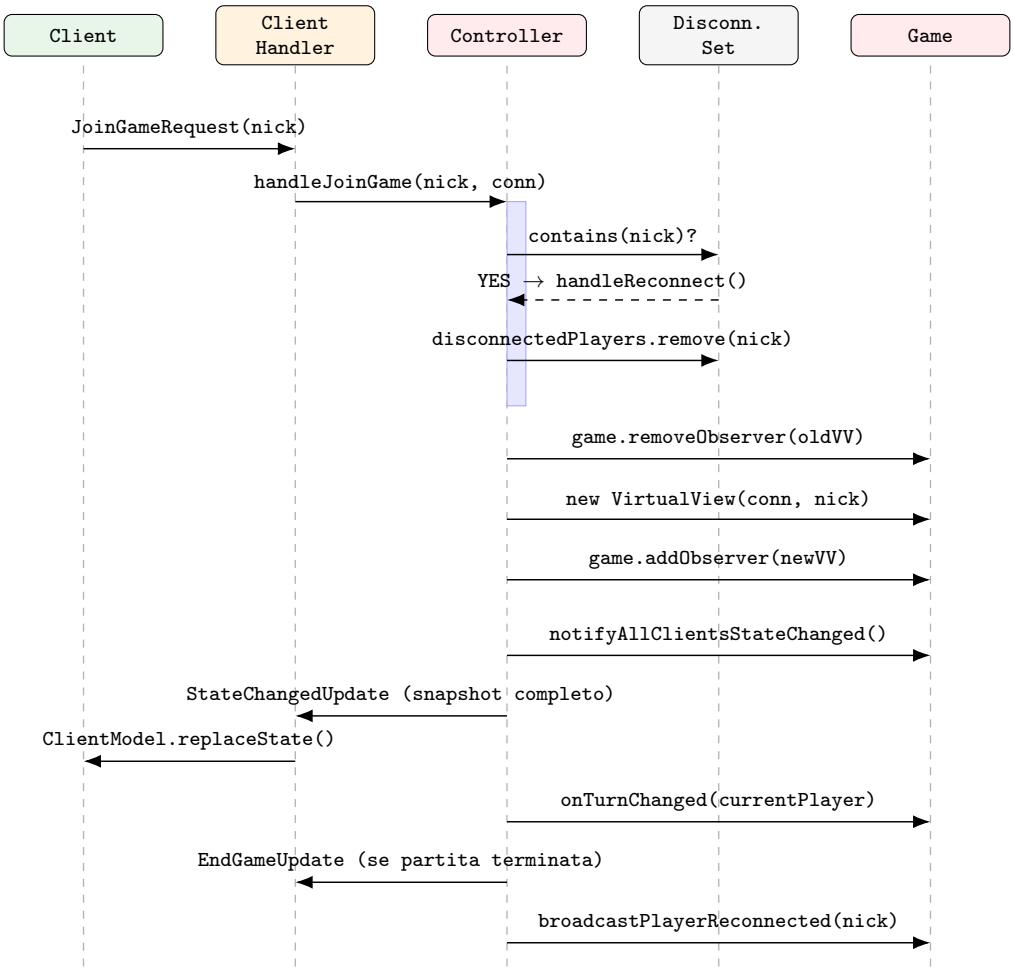


Figura 8: Riconnessione: il server ripristina lo stato tramite snapshot completo

6.2 Auto-Riconnessione Lato Client

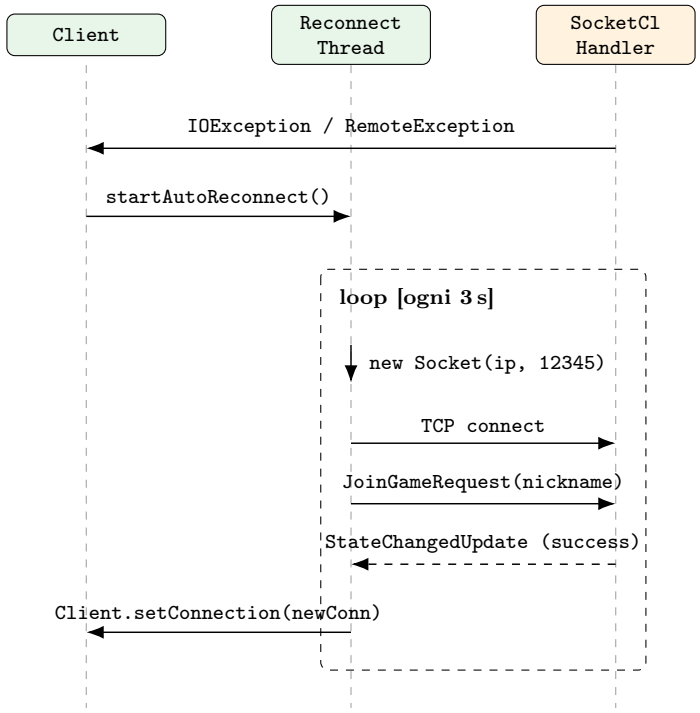


Figura 9: Auto-riconnessione client: retry con nuova connessione ogni 3 s

7 Recupero da Crash del Server

7.1 Backup JSON e Ripristino

Il server persiste l'intero oggetto `Game` su `backup.json` dopo ogni azione che altera lo stato. Il salvataggio è atomico tramite `Files.move(ATOMIC_MOVE)`: il file temporaneo viene completamente scritto prima di sostituire l'originale, garantendo che un crash durante la scrittura non corrompa il backup precedente.

Il backup usa Gson con `RuntimeTypeAdapterFactory` per la serializzazione polimorfica di `GameState`, `IEffect` e `TribeCard`. Il metodo `restoreReferences()` ri-collega i puntatori tra oggetti che Gson ricostruisce come copie separate.

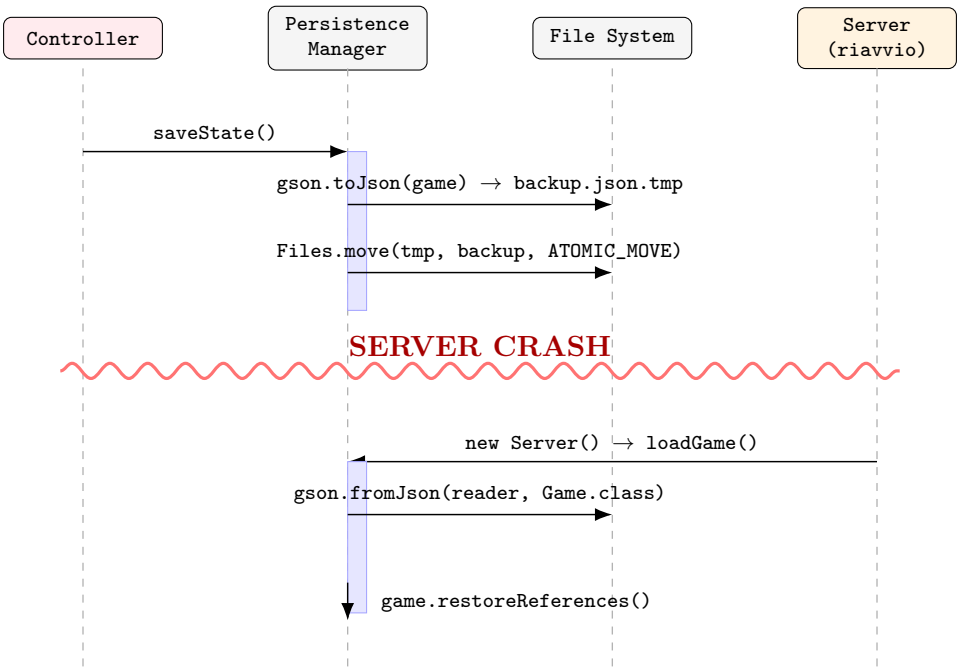


Figura 10: Backup atomico e recupero dopo crash del server

8 Interfacce di Connessione Lato Server

8.1 Gerarchia ClientConnection

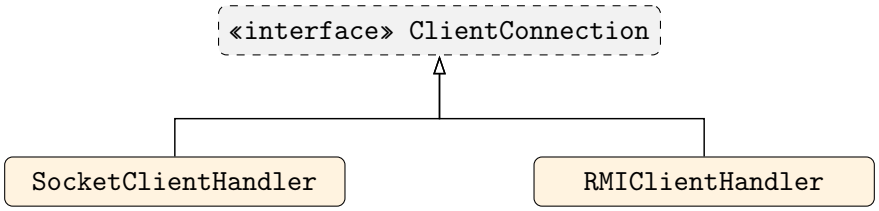


Figura 11: Gerarchia di ClientConnection lato server

8.2 API ClientConnection

Metodo	Descrizione
`notifyPlayerJoined(nickname)`	Nuovo giocatore entrato in lobby
`notifyTotemColor(nick, color)`	Colore totem scelto
`notifyTotemPlaced(nick, pos)`	Totem posizionato su offering tile
`notifyCardPicked(nick, row, idx, ...)`	Carta prelevata dal mercato
`notifyTotemReturned(nick, pos, food, aura)`	Totem restituito a fine pesca
`notifyEventResolved(name, stats, ...)`	Evento risolto
`notifyTurnChanged(nickname)`	Cambio turno
`notifyStateChanged(state, era, round, ...)`	Snapshot completo di stato
`notifyEndGame(winners, scores, num)`	Partita terminata
`notifyError(message)`	Errore o mossa non valida
`notifyPlayerDisconnected(nickname)`	Giocatore disconnesso
`notifyPlayerReconnected(nickname)`	Giocatore riconnesso
`notifyGameSuspended()`	Partita sospesa (1 attivo)
`notifyGameResumed()`	Partita ripresa (≥ 2 attivi)
`sendPing()`	Invia heartbeat al client
`sendLeaderboardUpdate(update)`	Risultato classifica
`disconnect()`	Chiude la connessione
`startHeartbeat()`	Avvia il ping scheduler

9 Interfacce di Connessione Lato Client

9.1 ServerConnection

Metodo	Descrizione
`joinGame(nickname)`	Connessione iniziale / rientro dopo crash
`setupGame()`	Segnala prontezza ad iniziare
`chooseTotem(color)`	Sceglie il colore del totem
`placeTotem(position)`	Posiziona il totem su una offering tile
`pickCard(row, index)`	Preleva una carta dalla fila indicata
`endTurn()`	Termina il proprio turno
`requestLeaderboard(numPlayers)`	Richiede la classifica filtrata
`disconnect()`	Chiude la connessione

9.2 Implementazioni

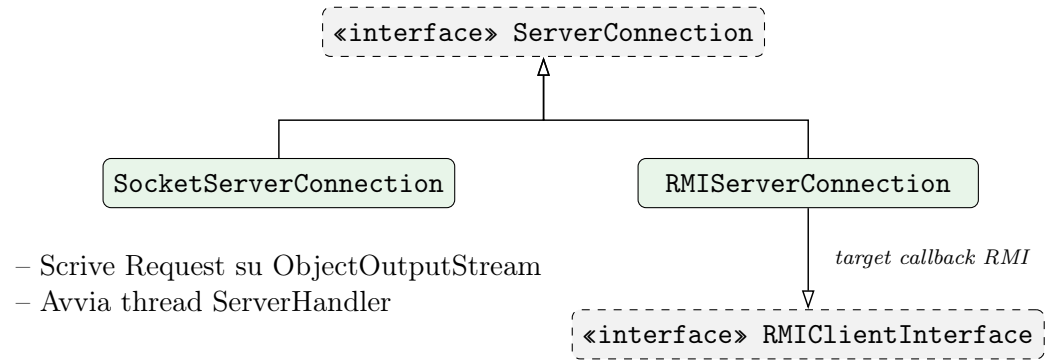


Figura 12: RMIServerConnection implementa anche RMIClientInterface per ricevere le callback

9.3 VirtualServer — Facciata per la View

`VirtualServer` recupera la connessione corrente dal singleton `Client` ad ogni chiamata (`Client.getInstance().getConnection()`), non al momento della costruzione. Dopo un'auto-riconnessione che sostituisce la `ServerConnection`, tutte le azioni future vengono automaticamente instradate sulla nuova connessione senza che la View debba essere aggiornata.

Metodo	Comportamento
`joinGame()`	`connection()` → `joinGame(nickname)`
`setupGame()`	`connection()` → `setupGame()`
`chooseTotem(color)`	`connection()` → `chooseTotem(color)`
`placeTotem(pos)`	`connection()` → `placeTotem(pos)`
`pickCard(row, idx)`	`connection()` → `pickCard(row, idx)`
`endTurn()`	`connection()` → `endTurn()`
`requestLeaderboard(n)`	`connection()` → `requestLeaderboard(n)`

10 Protocollo TCP Socket

10.1 Layout degli Stream

Ogni connessione TCP utilizza due `ObjectOutputStream` indipendenti su un singolo socket: uno per le Request (Client → Server) e uno per gli Update (Server → Client).

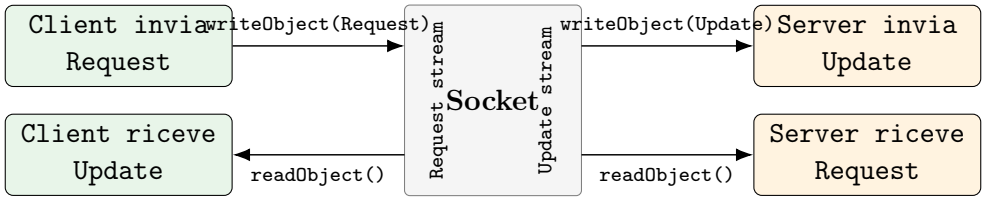


Figura 13: Doppio stream su singolo socket TCP

10.2 Dispatch Lato Server e Lato Client

Server (`SocketClientHandler.run()`):

1. **Intercettazione precoce** — `PongResponse` aggiorna `lastPongTime`; `JoinGameRequest` cattura il nickname.
2. **Dispatch polimorfico** — tutti gli altri sottotipi invocano `request.execute(controller, nick, this)`.

Client (`ServerHandler.handleUpdate()`):

1. **Intercettazione precoce** — `PingRequest` è risposto automaticamente con `PongResponse`, in modo trasparente alla UI.
2. **Dispatch polimorfico** — tutti gli altri sottotipi invocano `message.applyTo(clientModel)`.

11 Interfacce Remote RMI

11.1 Relazioni tra le Interfacce

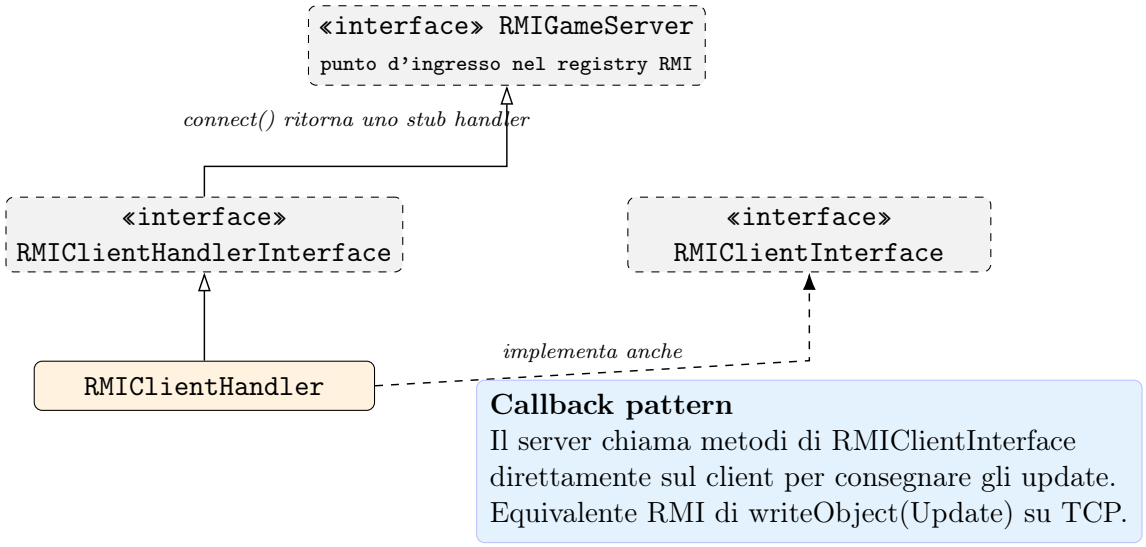


Figura 14: Relazioni tra le tre interfacce remote RMI

11.2 RMIGameServer — Punto di Ingresso

Metodo	Descrizione
`connect(RMIClientInterface clientStub)`	Registra il client, restituisce uno stub `RMIClientHandlerInterface`

11.3 RMIClientHandlerInterface — Azioni di Gioco

Metodo	Descrizione
`joinGameRequest(nickname)`	Connessione / rientro
`setupGameRequest()`	Prontezza ad iniziare
`chooseTotemColorRequest(color)`	Scelta colore totem
`placeTotemRequest(offerPosition)`	Posizionamento totem
`pickCardRequest(row, cardIndex)`	Pesca carta
`endTurnRequest()`	Fine turno
`getLeaderboardRequest(numPlayers)`	Richiesta classifica
`pong()`	Risposta heartbeat
`disconnect()`	Disconnessione esplicita

11.4 RMIClientInterface — Callback Client

Metodo	Descrizione
`onPlayerJoined(nickname)`	Nuovo giocatore in lobby
`onTotemColorChosen(nick, color)`	Colore totem scelto
`onTotemPlaced(nick, pos)`	Totem posizionato
`onTotemReturned(nick, food, aura)`	Totem restituito
`onCardPicked(nick, row, idx, food, magic, offering)`	Carta prelevata
`onTurnChanged(nickname)`	Cambio turno
`onStateChanged(state, era, round, players, ...)`	Snapshot completo
`onEndGame(EndGameUpdate)`	Partita terminata
`onErrorMessage(message)`	Errore
`onLeaderboardUpdate(update)`	Risultato classifica
`onPlayerDisconnected(nickname)`	Giocatore disconnesso
`onPlayerReconnected(nickname)`	Giocatore riconnesso
`onGameSuspended()`	Partita sospesa
`onGameResumed()`	Partita ripresa
`ping()`	Heartbeat server → client

12 Thread Safety

Controller Tutte le azioni di gioco tramite `executeCommand` sono sincronizzate sul monitor `game.getPlayers()`, impedendo mutazioni concorrenti da thread client multipli.

ClientModel Tutti i metodi di aggiornamento sono `synchronized`. Il thread `ServerHandler` (TCP) o il thread di callback RMI aggiornano il modello in modo sicuro rispetto al thread UI JavaFX.

Socket I/O Il metodo `writeToStream` in `SocketClientHandler` è `synchronized` per prevenire interleaving di scritture sullo stesso `ObjectOutputStream` da parte di observer multipli.

Heartbeat Il campo `lastPongTime` è `volatile long` per garantire visibilità immediata tra il thread di callback e il ping scheduler.

Stato finale I campi `finalWinners`, `finalScores` e `finalNumPlayers` del `Controller` sono `volatile` per rendere visibile il risultato catturato a fine partita al thread che gestisce la riconnessione.

13 Riferimenti ai File Sorgente

File	Ruolo
`network/server/Server.java`	Entry point: ServerSocket TCP + registry RMI
`network/server/SocketClientHandler.java`	Thread TCP per singolo client
`network/server/RMIServerImpl.java`	Implementazione server RMI
`network/server/RMIClientHandler.java`	Handler RMI per singolo client
`network/server/VirtualView.java`	Osservatore Game → broadcast di rete
`network/server/ClientConnection.java`	Interfaccia astrazione trasporto server
`network/server/PersistenceManager.java`	Backup JSON per crash recovery
`network/client/Client.java`	Singleton hub del client
`network/client/ServerConnection.java`	Interfaccia astrazione trasporto client
`network/client/SocketServerConnection.java`	Implementazione TCP client
`network/client/RMIServerConnection.java`	Client RMI + target callback
`network/client/ServerHandler.java`	Thread lettura background TCP
`network/client/ClientModel.java`	Stato locale lato client
`network/client/VirtualServer.java`	Facciata azioni per UI
`network/messages/ClientMessage.java`	Interfaccia marcatore radice
`network/messages/Request.java`	Interfaccia sealed Request
`network/messages/Update.java`	Interfaccia sealed Update
`network/rmi/RMIGameServer.java`	Interfaccia remota d'ingresso RMI
`network/rmi/RMIClientInterface.java`	Interfaccia callback client RMI
`network/rmi/RMIClientHandlerInterface.java`	Interfaccia handler server RMI